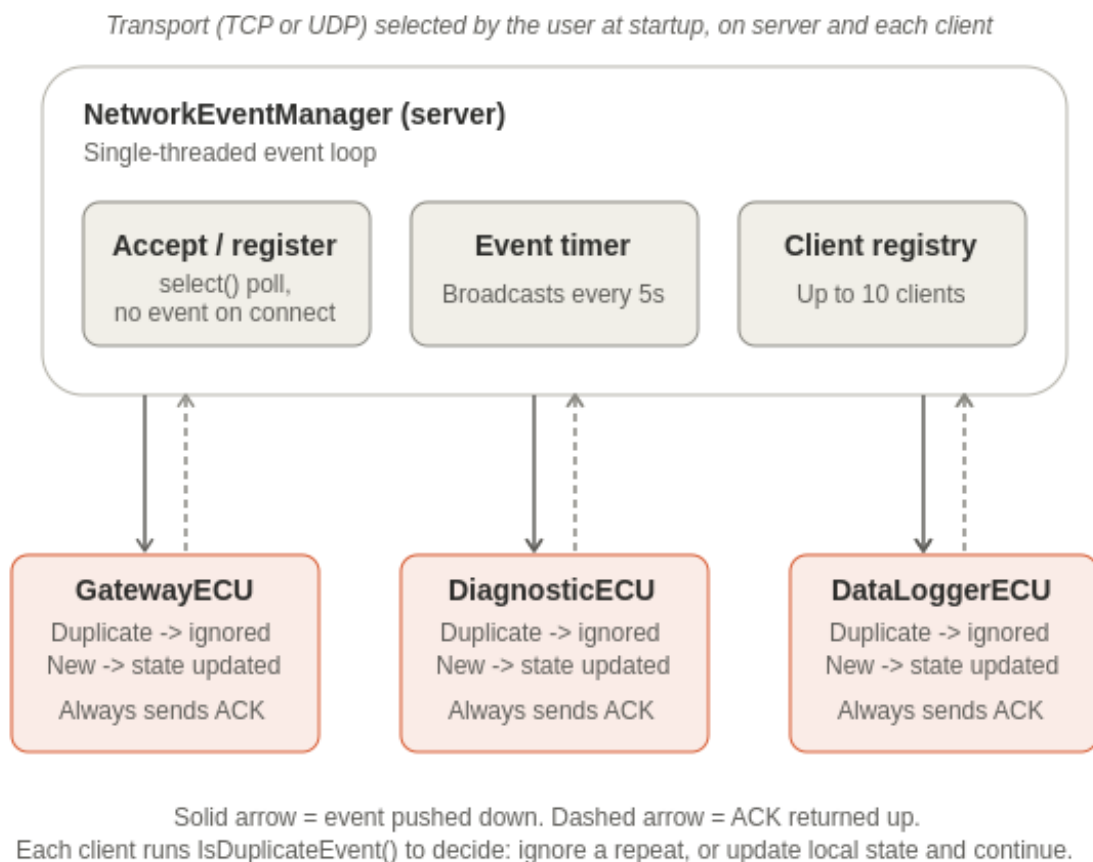


Automotive Network Monitor System

1. Overview

A lightweight client-server event update system in C over TCP/IP. One server (NetworkEventManager) generates events and pushes them to multiple connected clients (ECUs): GatewayECU, DiagnosticECU, and DataLoggerECU. Each client checks whether an event was already processed, ignores duplicates, updates its local state for new events, and always sends an acknowledgement back to the server. Transport (TCP or UDP) is selectable at runtime by both the server and each client.

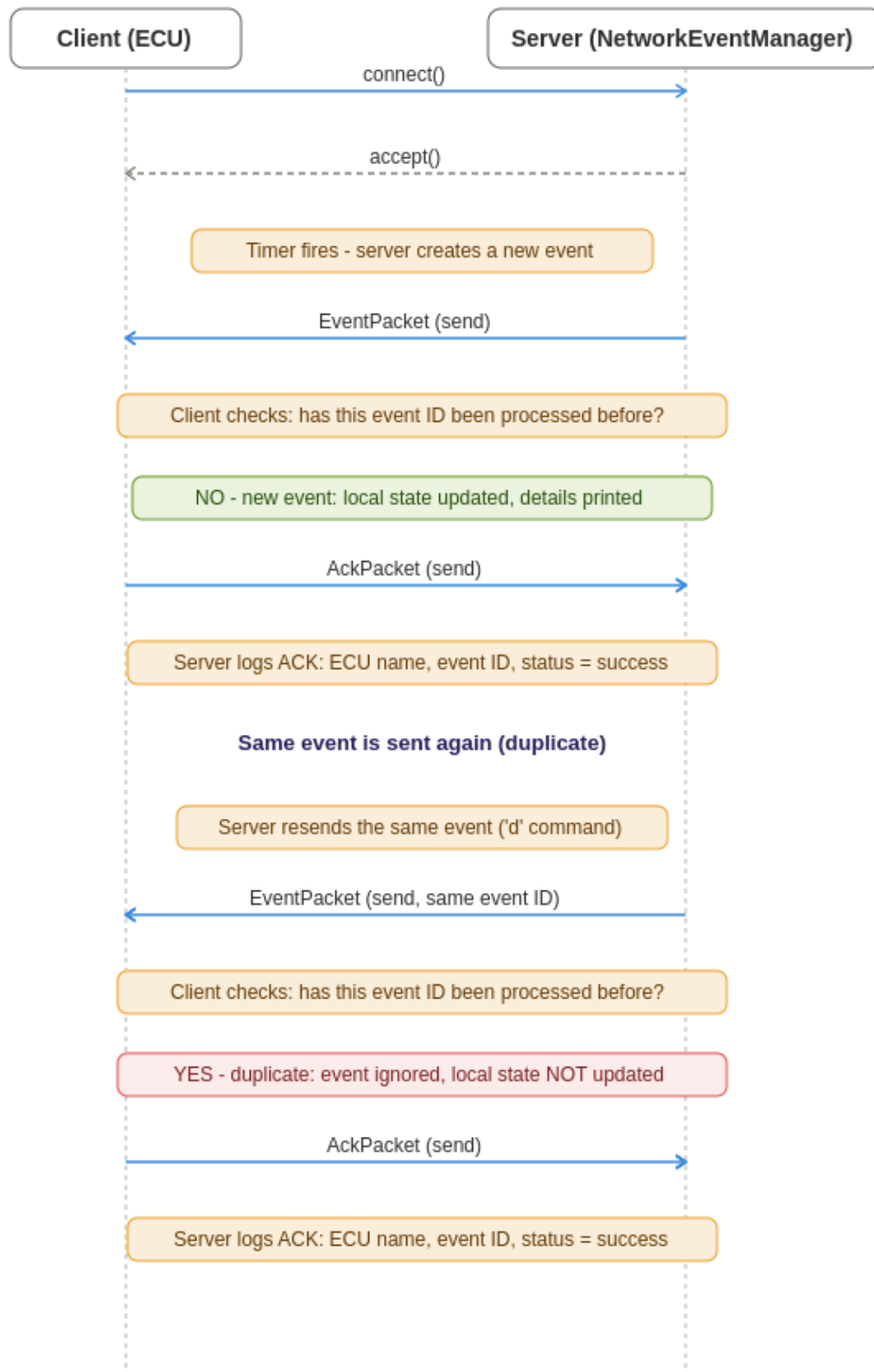
2. High-Level Architecture



- Transport is chosen independently by the server and each client at startup. Event generation is decoupled from client
- connections: a timer in the server's main loop broadcasts a new event on a fixed interval, regardless of when or how
- many clients are connected. Everything on the server (accepting/registering clients, the event timer, and manual
- keyboard commands) runs in a single thread, using `select()` to poll the network without blocking.

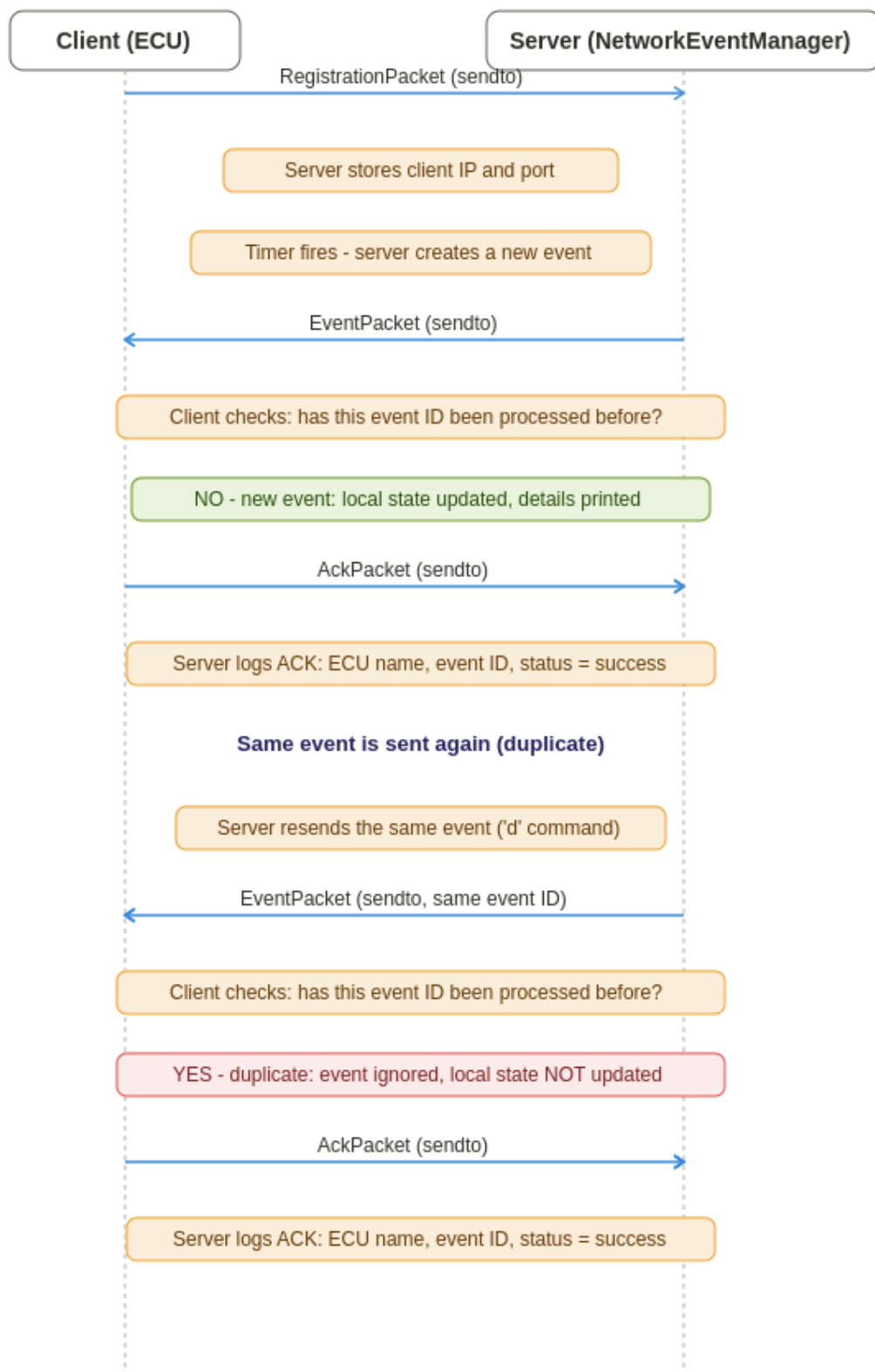
3. SequenceDiagram – TCP

Shows the complete cycle on a TCP connection: connect, an automatically generated event being processed as new, and the same event being resent to show the duplicate path.



4. Sequence Diagram – UDP

Same complete cycle over UDP: registration in place of connect, an automatically generated event processed as new, and the same event resent to show the duplicate path.



5. Key Design Decisions

- Event generation is decoupled from client connections - a timer drives event broadcast, independent of when or how many clients are connected.
- Single-threaded event loop with `select()` - accepting clients, broadcasting, and reading commands run in one loop, avoiding multi-thread coordination issues on shared sockets.
- History-based duplicate detection - each client tracks its last 20 processed event IDs, not just the most recent one.
- Manual trigger commands - allow sending a specific event type or resending the last event on demand, for reliable live demonstration.

6. What's Implemented

- TCP communication - server socket, `accept()` for incoming client connections, `send()/recv()` for events and ACKs.
- UDP communication - connectionless; each client first sends a `RegistrationPacket` so the server can learn its address; events use `sendto()`, ACKs use `recvfrom()`.
- Transport selection - both server and client choose TCP or UDP at startup.
- Readable event data - event type and severity shown as names (e.g. "Engine Start", "HIGH") instead of raw numbers, along with event ID and source ECU.
- Duplicate detection - each client keeps a history of the last 20 processed event IDs, ignores repeats, but still sends an ACK for them.
- ACK mechanism - every event is acknowledged back to the server with ECU name, event ID, and status, for both TCP and UDP.
- Multiple client support - server tracks up to 10 connected clients and broadcasts to all of them.
- Server-driven event generation - a timer generates a new event automatically, independent of client connections.
- Manual demo commands - the server can send a specific event type, or resend the last event, for live demonstration of duplicate handling.

7. Known Limitations

- Sequential broadcast - the server processes clients one at a time rather than in parallel; a slow client can delay delivery to others.
- In-memory state only - client list and event history reset on restart; no persistence layer.
- Fixed-size duplicate window - each client tracks only its last 20 event IDs, sufficient for a demo but not designed for long-running scale.